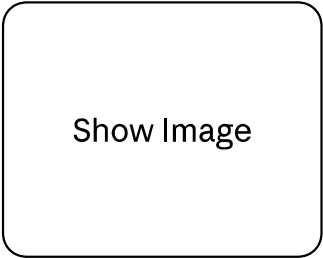


Handwritten Digit Recognition

CNN trained on MNIST that classifies handwritten digits (0–9) from uploaded images. Built with TensorFlow and deployed as a Streamlit web app.

Final model accuracy: 99.19% on MNIST test set

Demo



Show Image

What this does

Upload a photo of any handwritten digit and the model predicts it in real time with a confidence score and full probability distribution across all 10 digits.

The app has three tabs:

- **Predict** — upload an image, get a prediction
 - **Training** — accuracy and loss curves across all 15 epochs
 - **About** — model architecture and how to run it
-

Model Architecture

```
Input (28×28×1)
  → Conv2D(32) → MaxPooling → Dropout(0.25)
  → Conv2D(64) → MaxPooling → Dropout(0.25)
  → Conv2D(128) → Dropout(0.25)
  → Flatten
  → Dense(256) → Dropout(0.5)
  → Dense(10, softmax)
```

Improvements over the base tutorial model:

- Added a third Conv2D layer (128 filters) for deeper feature extraction
 - Dropout at every stage to reduce overfitting
 - Data augmentation — random rotation, zoom, and shifts on training images
 - 15 epochs instead of 5
 - Smart preprocessing pipeline that auto-crops and centers real photos before inference
-

Training Results

Metric	Value
Train Accuracy	97.05%
Validation Accuracy	99.19%
Validation Loss	0.0196
Epochs	15
Dataset	MNIST (60k train / 10k test)

Show Image

A note on real-world performance

The model hits 99.19% on the MNIST test set but can misclassify real handwritten digits that deviate from the training distribution. A stylistic 5 with a closed bottom loop, for example, may be read as an 8. This is a known domain gap between clean benchmark data and real-world inputs — benchmark accuracy and deployment accuracy are not the same thing.

Run locally

```
bash
```

```
# Clone the repo
git clone https://github.com/garvranaaa/handwriting-digit-recognition
cd handwriting-digit-recognition

# Create virtual environment
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows
source .venv/bin/activate      # Mac/Linux

# Install dependencies
pip install -r requirements.txt

# Train the model (run once – takes ~10-15 mins on CPU)
python train_model.py

# Launch the app
streamlit run app.py
```

Stack

- **TensorFlow / Keras** — model training
- **Streamlit** — web app
- **NumPy / Pillow** — image preprocessing
- **Matplotlib** — training visualizations

Built by

Garv Rana · EE Undergrad · Delhi Technological University

[GitHub](#) · [LinkedIn](#)